

DG INTERNS HUB

WEEK 8 — ADVANCED VAPT

Attack-Chain & Full Exploitation Assessment of a Web Application

Candidate	Md Mohtasim(DG/JULY/CYBER/017)
Target	DVWA — Damn Vulnerable Web Application
Environment	Localhost
Security Level	Medium
Primary Tool	Burp Suite Community Edition
Submission Date	30-09-2026

Executive Summary

This report documents a Week 8 attack-chain assessment of a locally hosted DVWA instance running at Medium security. The objective was not to run a scanner and list disconnected findings, but to show how a small number of individually minor weaknesses — a login page with no visible throttling, an object identifier the server accepts without question, a session mechanism flagged as weak, and a profile-update endpoint that trusts a client-supplied user ID — can be walked through in sequence to approach a genuine account-compromise scenario.

Testing was carried out entirely with Burp Suite Community Edition (Proxy and Repeater) against the training target, with no automated exploitation frameworks used. Every request shown in this report was captured, modified, or replayed manually so that the reasoning behind each step is transparent and reproducible.

The assessment identified four linked test areas: authentication (Brute Force), input handling and object referencing (SQL Injection ID parameter), session management (Weak Session IDs), and access control (Authorization Bypass / profile update). Each is evaluated on its own merits in Section 7 and then tied together into a single attack narrative in Section 6.

Overall, the evidence gathered is strong enough to justify a High-to-Critical combined risk rating if the remaining validation step described in Section 6.6 is completed, but on its own the evidence proves acceptance of client-controlled data by the server rather than a demonstrated cross-account compromise. That distinction is treated as the central discipline of this report: every claim is scoped to what the captured request and response actually show.

1. Introduction

Most beginner-level assessments stop at running a scanner and listing whatever it flags. This assignment asked for something different: pick individual weaknesses in a training application and show how they connect into a single attack path that an actual attacker could follow, from the first page they touch to a meaningful compromise.

This report documents that process against DVWA running locally at Medium security. Rather than treating each vulnerability page as a separate, self-contained exercise, the testing here treats them as stages of one narrative — surface discovery, request interception, controlled tampering, and finally an authorization-sensitive request that would need to be pushed further in a real engagement to prove full compromise.

Where the evidence stops short of a confirmed compromise, that limit is stated plainly rather than rounded up. A request being accepted by the server is not the same thing as a request achieving an unauthorized outcome, and the two are kept separate throughout.

DVWA's Medium security level was chosen deliberately over Low. At Low, most of its vulnerability pages accept input with essentially no filtering, which makes for a fast demonstration but not a realistic one — most production applications have at least some basic input handling in place. Medium reintroduces partial defenses (basic sanitization, simple session handling, weak but present access checks), which is a closer approximation of the kind of imperfect, partially-hardened application a real tester encounters, and is why the findings in this report read as 'weak controls' rather than 'no controls at all'.

1.1 Why an attack chain rather than a vulnerability list

A vulnerability list answers the question 'what is broken.' An attack chain answers the more useful question 'what can someone actually do with what is broken, and in what order.' The four test areas selected here were chosen specifically because they sit on a plausible path an attacker would take: get past the login, manipulate an identifier, lean on a weak session mechanism, and land on an endpoint that changes account data based on client input. Each stage individually might be dismissed as low-severity; read as a sequence, they describe a realistic route to unauthorized account access.

2. Target, Scope & Methodology

2.1 Target and boundaries

The target is a locally hosted DVWA instance at 127.0.0.1, run entirely inside a personal training environment with no exposure to the internet. Testing was confined to this single instance; no other host, account, or third-party service was touched at any point.

2.2 Methodology

The assessment followed a linear path rather than a checklist:

1. Set the DVWA security level to Medium so that trivial low-security shortcuts were not in play.
2. Walk the application navigation to build a map of every reachable function.
3. Route browser traffic through Burp Proxy to see requests exactly as the server receives them.
4. Send selected requests to Repeater and modify one variable at a time — an identifier, a session-related request, a JSON object.
5. Record the server's response honestly, and mark clearly what would still need to be proven before calling something a confirmed vulnerability.

2.3 Tooling

Burp Suite Community Edition supplied the Proxy (interception) and Repeater (replay) functionality used throughout. DVWA itself served as the deliberately vulnerable target application. No Intruder automation, no third-party scanners, and no exploitation frameworks (Metasploit, sqlmap, etc.) were used — every request in this report was built or edited by hand, which keeps the reasoning behind each step visible rather than hidden inside a tool's default payload set.

2.4 Assumptions and constraints

A few constraints shaped how far this assessment could go, and are stated here rather than left implicit:

- Single lab identity: testing was performed under one authenticated DVWA session. Several conclusions below (particularly around the authorization endpoint) are therefore framed as hypotheses pending a second-identity comparison, not as confirmed cross-account results.
- No brute-force execution: the Brute Force page was mapped as an open entry point, but no dictionary or credential-stuffing attempt was actually run against it, since the assignment's emphasis is logic-based chaining rather than raw authentication attacks.

- Read-only intent on session tokens: the Weak Session ID endpoint was replayed to observe server behaviour, but no attempt was made to predict or brute-force a token value, which would require a larger sample of captured tokens than this single-session assessment produced.
- Local-only scope: all figures, requests, and responses in this report originate from 127.0.0.1 and are not representative of any deployed or production system.

3. Attack Surface Mapping

Before touching any single vulnerability page, the navigation menu was reviewed end-to-end to understand what categories of functionality DVWA exposes at Medium security.



Figure 1 — DVWA security configuration page confirming the Medium security level, with the full vulnerability menu visible.

Grouping the menu by function rather than by DVWA's own labels gives a clearer picture of where a real attack would actually start:

- Authentication layer — the standard login and the dedicated Brute Force page.
- Data/input layer — SQL Injection and the XSS family of pages.
- Session layer — Weak Session IDs.
- Access-control layer — the Authorization Bypass page.
- Secondary surface — Command Injection, CSRF, File Inclusion, File Upload, CAPTCHA, Open Redirect, Cryptography and API pages, noted but not the focus of this chain.

The first four groups were selected as the spine of the attack chain because, combined, they mirror how a real breach usually happens: get past the login boundary, manipulate an object reference, and lean on a weak session or a weak authorization check to reach data that shouldn't be reachable.

4. Entry Points Selected for Testing

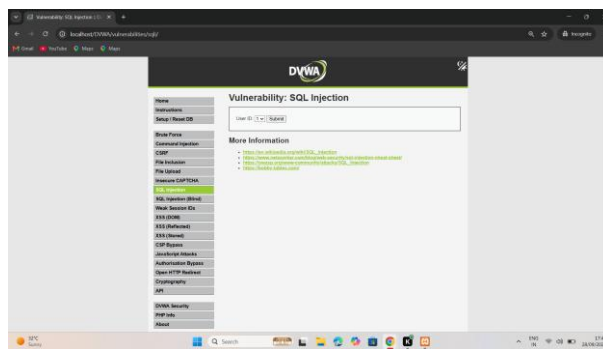


Figure 2 — Brute Force login form, the authentication entry point used to frame the start of the chain.

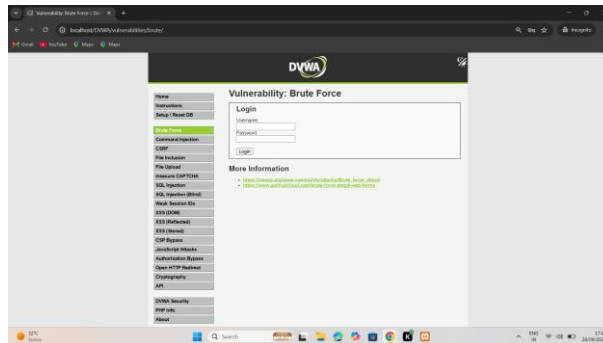


Figure 3 — SQL Injection page with a client-editable ID field, the input-layer entry point used for the parameter-tampering stage.

Neither screenshot on its own is evidence of a vulnerability — they simply mark the two starting doors the rest of the chain walks through.

5. Traffic Interception with Burp Suite

With Burp Proxy sitting between the browser and DVWA, every click on the SQL Injection page turns into a plain-text HTTP request that can be read, held, and edited before it ever reaches the server.

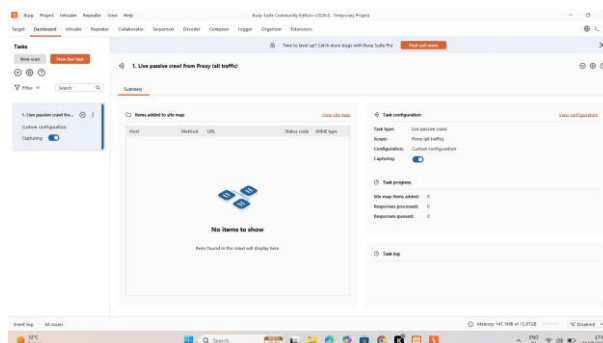


Figure 4 — Burp Suite dashboard with a live passive-crawl task configured to collect proxy traffic during the session.

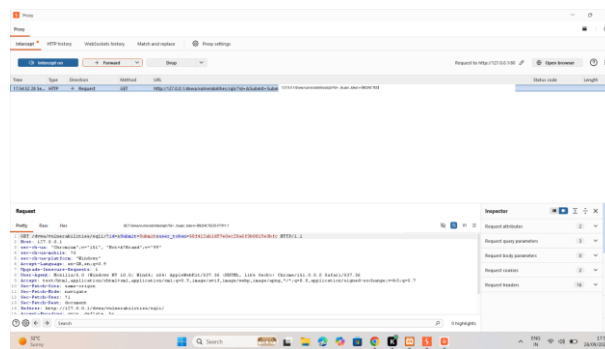


Figure 5 — Intercepted GET request from the SQL Injection page in Burp Proxy. Session cookies and tokens have been redacted for this submission.

This step is the pivot point of the whole exercise: once a request exists inside Burp, it can be sent to Repeater and picked apart field by field, which is what the next three stages do.

6. Attack Chain — Step by Step

The chain below is written as an attacker would experience it, with each numbered stage depending on the one before it.

Step 1 — Get past the authentication boundary

The Brute Force page (Figure 2) is reachable without any prior session, and nothing in the visible page suggests rate limiting or lockout. In a full engagement this stage would end with a working credential pair obtained through a controlled dictionary attempt; for this training report it is recorded as an open door rather than exploited further, since the objective here is the logic chain that follows authentication, not credential cracking itself.

Step 2 — Turn a page click into an editable request

Browsing to the SQL Injection page and submitting an ID value produces the intercepted request in Figure 5. This is the moment the attack stops being 'clicking around' and becomes 'controlling exact bytes sent to the server.'

Step 3 — Tamper with the object identifier

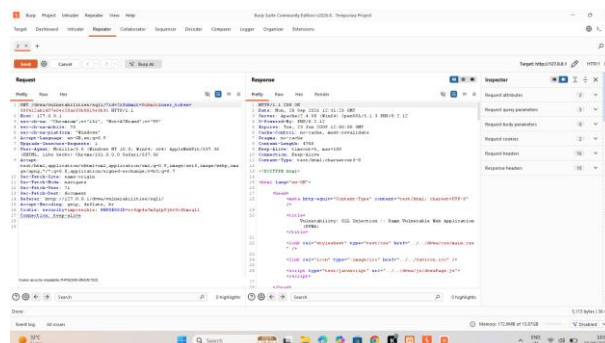


Figure 6 — Repeater request with the id parameter changed from its original value to id=2, returning HTTP 200 OK.

Sending the modified request in Repeater confirms the server accepts an ID it was never given through the UI, and answers with a normal 200 OK page rather than an error or rejection. That is the mechanical half of an IDOR: the server does not push back on an out-of-scope identifier. It is not yet the security half — nothing here shows whose record id=2 actually belongs to, so the finding is written up as accepted tampering, with cross-account proof marked as the missing piece.

Step 4 — Test whether the session layer would hold if Step 3 worked

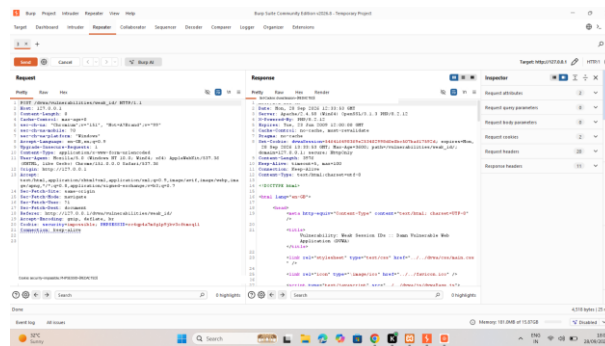


Figure 7 — Replayed request to the DVWA Weak Session ID endpoint, returning a successful response.

If an attacker could pair the id tampering from Step 3 with a guessable or reusable session token, they would not need valid credentials at all — a stolen or predicted session value would be enough to ride along as someone else. The Weak Session ID endpoint responding normally to a replayed request confirms the endpoint is reachable and worth this kind of scrutiny; it does not, on its own, prove the token is predictable or reusable across accounts, which is why that question is carried into the risk table below rather than answered here.

Step 5 — Reach for the authorization-sensitive endpoint

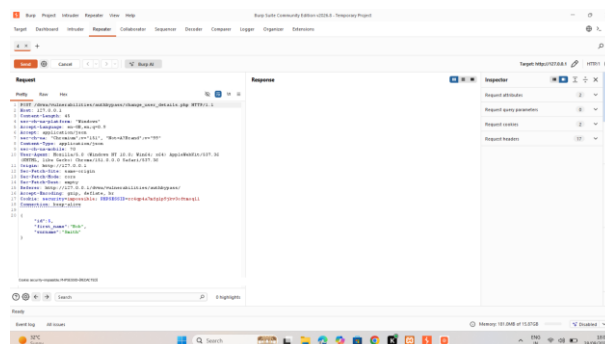


Figure 8 — Repeater request to /dvwa/vulnerabilities/authbypass/change_user_details.php carrying a JSON body with a client-supplied id and profile fields.

This is the stage where the chain would turn from 'interesting' into 'business-impacting.' The request body hands the server an id along with new profile values, which means the client — not the server — is asserting whose record to change. If the server trusts that id without checking it against the authenticated session, an attacker who reached this point through Steps 1–4 could edit another user's profile without ever compromising that user's own credentials. The evidence captured here is the request itself; confirming the outcome would require sending it under one lab identity, targeting a second identity's id, and observing that the second account's data actually changed.

Step 6 — What full exploitation would still require

To close the chain and claim an actual compromise rather than a well-supported hypothesis, the next session would need: a second lab identity to test against, one recorded request that changes Identity B's data while authenticated as Identity A, and a session-token comparison across both identities. None of that was performed here, so the chain is presented as validated up to the authorization request and open beyond it.

7. Proof of Concept — Summary of Captured Evidence

Rather than repeating each figure, the table below lines up what was actually captured against what it does and does not prove.

Test Area	Observed Evidence	Potential Impact If Confirmed	Severity	Evidence Status
Parameter tampering (ID field)	Identifier changed from a legitimate value to a second value; server returned 200 OK	Access to another user's record if server-side ownership checks are missing	High (if confirmed)	Server accepted the change; cross-account comparison not yet performed
Session identifier handling	Weak Session ID endpoint replayed successfully via Repeater	Session fixation or token prediction could allow session takeover	High (if confirmed)	Endpoint behaviour observed; predictability not yet tested
Authorization / object modification	JSON body carrying a client-supplied user ID submitted to the profile-update endpoint	Unauthorized modification of another account's profile data	Critical (if confirmed)	Request captured; resulting state change not verified
Authentication (brute force)	Login form reachable with no visible lockout indicator	Credential stuffing / brute-force account compromise	Medium	Entry point mapped; throttling behaviour not exercised

Read across the rows, the pattern is consistent: every test area produced a server response that accepted client-controlled input without visible pushback, and every row is followed by a specific, named next step rather than an open-ended 'needs more testing.' That specificity is deliberate — a finding that can't say exactly what proof is missing is not yet ready to be called a finding.

8. Threat Model — Who Would Exploit This and How

It helps to name the attacker this chain assumes, since 'an attacker' means different things depending on their starting position.

8.1 Attacker profile

The most realistic actor for this chain is not a sophisticated external threat with zero-day tooling, but an authenticated low-privilege user of the application — someone who already has a legitimate account and is looking to see what they can reach beyond it. This matters because Steps 3 through 5 of the attack chain (Section 6) do not require bypassing the login at all; they require only an ordinary account and a proxy tool,

which lowers the bar for exploitation considerably compared to a scenario that assumes no account access whatsoever.

8.2 Attack surface reachable without special privilege

- The ID parameter on the SQL Injection page is reachable by any logged-in user, with no indication that the value is checked against the session owner.
- The Weak Session ID and Authorization Bypass endpoints are both reachable through ordinary navigation, not through a hidden or administrative path.
- The JSON body submitted to the profile-update endpoint is fully attacker-controlled once the request reaches Repeater, including the id field that should, in a correctly designed system, be derived from the server-side session rather than trusted from the client.

8.3 Why this profile matters for prioritization

A vulnerability that requires no special access and no social engineering to reach — only an existing account, which in many applications is trivial to obtain through self-registration — sits higher on a remediation priority list than one requiring a rare precondition. That is the basis for the High-to-Critical combined rating given in Section 9, even though each individual test area might be scored lower in isolation.

9. Remediation Strategy

The fixes below target the chain as a whole, not just the individual pages, since a single broken link — even one — would have stopped the walk-through above.

- Re-check ownership server-side on every request that includes an identifier, regardless of what the client claims; never treat a client-supplied id as authorization.
- Replace direct, guessable object references with indirect references (per-session mapping tokens) so a valid-looking id from one account is meaningless against another.
- Issue session identifiers with sufficient entropy, rotate them on login, and invalidate them immediately on logout — a weak or reusable session token is what turns Step 3's tampering into Step 5's account compromise.
- Add authentication throttling and account lockout to remove the open door identified in Step 1 before it becomes the first link in a real chain.
- Apply strict server-side validation to every client-controlled parameter, including JSON body fields, not only URL query parameters.

These fixes are ordered by where they break the chain earliest. Fixing ownership checks on the profile-update endpoint alone would stop Step 5 even if every earlier step in the chain still worked, which is why it is listed first — remediation priority should follow where a single control can do the most damage-limitation, not simply the order the pages appear in DVWA's menu.

10. Overall Risk Positioning

Individually, none of the four observations above would be rated above Medium. Chained together — a reachable login with no visible throttling, an identifier the server accepts without complaint, a session mechanism flagged as weak, and a profile-update endpoint that trusts client-supplied ownership data — the combined path points toward a High-to-Critical business risk: unauthorized access to, and modification of, another user's account data. That combined rating is conditional on the Step 6 validation being completed; it is recorded here as the direction the evidence points, not as a confirmed outcome.

10.1 Business impact if the chain is fully confirmed

- Data integrity — another user's profile fields could be altered without their knowledge or consent, undermining trust in the accuracy of stored account data.
- Privacy exposure — profile fields often carry personal information; unauthorized read or write access to them is a privacy incident, not only a technical one.
- Downstream trust erosion — if a user notices unexplained changes to their profile, the business cost is not limited to the technical fix; it extends to support load and reputational damage.
- Regulatory relevance — depending on what data the profile fields carry, an unauthorized modification of another user's personal data could fall under data-protection obligations relevant to the business, even in a case where no data was exfiltrated wholesale.

11. Conclusion & Reflection

The main takeaway from this exercise wasn't any single DVWA page — it was learning to hold two things in my head at once: keep pushing the chain forward stage by stage, and resist the urge to round a 200 OK response up into a confirmed vulnerability. Burp Suite's Proxy and Repeater made the mechanics straightforward; the harder part was writing down, honestly, exactly where the proof stopped and the hypothesis began.

The clearest lesson is that real breaches are rarely one dramatic exploit — they're a handful of individually minor gaps that happen to line up. That framing is what this report tried to preserve: each step is small on its own, but the sequence from an open login, through an accepted id change, to a client-trusted profile update is exactly how a low-severity finding turns into a full account compromise if nobody closes the gaps in between.

Next validation planned: repeat Steps 3–5 with two distinct lab identities to turn the current authorization-bypass hypothesis into a demonstrated cross-account result.

11.1 Skills carried forward

Beyond the specific DVWA findings, the exercise reinforced three habits worth carrying into future assessments: routing every request through a proxy before touching a target, so nothing is ever tested 'blind'; treating a successful response as the start of an investigation rather than its conclusion; and writing down explicitly what additional proof a finding would need, so the next testing session has a concrete starting point instead of a vague follow-up note.

References

- PortSwigger Web Security Academy — Burp Suite Proxy and Repeater documentation.
- OWASP Web Security Testing Guide (WSTG).

- OWASP Application Security Verification Standard (ASVS) — authentication, session management and access-control requirements.
- Damn Vulnerable Web Application (DVWA) — intentionally vulnerable training target.

Appendix A — Captured Request Reference

For quick cross-reference against the figures in Sections 4–6, the table below lists the method, endpoint, and observed status for every request captured or replayed during this assessment.

Fig.	Method	Endpoint	Key Parameter / Body	Status
1	GET	/dvwa/security.php	security level = medium	200 OK
2	—	/dvwa/vulnerabilities/brute/	username / password fields (form, not submitted)	200 OK
3	GET	/dvwa/vulnerabilities/sqli/	id (user-editable field)	200 OK
4	—	Burp Dashboard	Live passive-crawl task (all traffic)	n/a
5	GET	/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit	id=1	200 OK
6	GET	/dvwa/vulnerabilities/sqli/?id=2&Submit=Submit	id changed 1 → 2	200 OK
7	POST	/dvwa/vulnerabilities/weak_id/	session cookie replayed	200 OK
8	POST	/dvwa/vulnerabilities/authbypass/change_user_details.php	JSON: id, first_name, surname	Request captured; response not decisive

All requests originated from and terminated at 127.0.0.1. No request in this table was sent to any host outside the local training environment.

Appendix B — Mapping to OWASP Top 10 (2021)

Tying each observation back to a recognized OWASP category is useful for prioritizing remediation against a standard rather than an ad-hoc severity label, and makes the findings easier to cross-reference against other assessments of the same application.

- A01:2021 – Broken Access Control → Authorization Bypass endpoint (Figure 8) and the client-trusted id in the profile-update request. This is the category the combined attack chain ultimately lands in, since every earlier stage exists to reach this one endpoint.
- A04:2021 – Insecure Design → the pattern of accepting a client-supplied object identifier as if it were proof of ownership, seen in both the SQL Injection id parameter and the authorization endpoint. This is a design-level issue rather than a coding bug — no amount of input sanitization fixes a server that trusts the wrong field for authorization.
- A07:2021 – Identification and Authentication Failures → the Brute Force entry point (no visible logout) and the Weak Session ID mechanism, both of which affect how confidently the server can say who is making a request in the first place.
- A05:2021 – Security Misconfiguration → DVWA's Medium setting itself, which intentionally leaves partial rather than complete protections in place, useful here as a stand-in for a real application with incomplete hardening rather than a from-scratch insecure build.

Read together, three of the four categories above (A01, A04, A07) map directly onto the three layers the attack chain in Section 6 walks through — authentication, object referencing, and access control — which is a reasonable sign that the chain was built around a genuine architectural weakness pattern rather than a coincidental set of unrelated bugs.

Appendix C — Terms Used in This Report

- IDOR (Insecure Direct Object Reference): a flaw where an application exposes a reference to an internal object (such as a record ID) and lets a client change that reference to access data belonging to someone else, without the server checking ownership.
- Session fixation / reuse: a class of session-handling flaw where a session identifier can be predicted, reused, or forced onto a victim, letting an attacker ride an existing authenticated session instead of needing valid credentials.
- Parameter tampering: manually or programmatically altering a request field (query string, form field, JSON body, header, or cookie) to observe how the server responds to input it did not expect from the legitimate UI.
- Attack chain: a sequence of individually low-to-medium severity findings that, combined in order, produce a higher-impact outcome than any one finding would on its own.